SpiderBoost and Momentum: Faster Stochastic Variance Reduction Algorithms

Zhe Wang, Kaiyi Ji, Yi Zhou, Yingbin Liang and Vahid Tarokh

NeurIPS 2019

List item 52 / Ref 80 | Tags: S, D / B | Tier: Skim

The paper in one minute

Question: Can variance-reduced stochastic optimization use a large constant step, handle nonsmooth regularizers and benefit from momentum?
Model/Method: SpiderBoost uses a SPIDER/SARAH-style recursive gradient estimator with periodic full-gradient resets, plus proximal and momentum variants.
Finding: The method attains near-optimal first-order gradient-oracle complexity with constant-order step size; proximal and momentum variants preserve that order, while experiments show speed improvements.
Why we care: Better stochastic gradient estimates accelerate the *first-order* phase of nonconvex optimization, though this paper does not itself establish saddle-escape guarantees.

1. Why variance reduction matters

For a finite-sum loss $f(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x)$, SGD uses one sampled gradient $\nabla f_i(x)$. Its expectation is correct, but its noise can make small-gradient optimization slow. The SPIDER/SARAH family remembers a previous gradient estimate and corrects it using matched samples at two nearby iterates.

The recursive estimator (paper Eq. 1)

At the beginning of an epoch, reset $v_k = \nabla f(x_k)$. Otherwise, on the same mini-batch S_k for both points, set

$$v_k = v_{k-1} + \frac{1}{|S_k|}\sum_{i\in S_k}\big(\nabla f_i(x_k) - \nabla f_i(x_{k-1})\big).$$

If x_k moves only a little, each *difference* is often less variable than a raw stochastic gradient. However, v_k is not an automatically unbiased estimator conditioned on the entire trajectory; the proof controls accumulated estimation error within each epoch.

Worked example: the difference cancels sample-specific offsets

Take $f_1(x) = \frac{1}{2}(x-1)^2$ and $f_2(x) = \frac{1}{2}(x+1)^2$. The full gradient is $f'(x) = x$. Let $x_{k-1} = 1.5$, $v_{k-1} = 1.5$ and $x_k = 2$. Whichever sample is chosen,

$$f_i'(2) - f_i'(1.5) = 0.5, \quad v_k = 1.5 + 0.5 = 2 = f'(2).$$

Yet the raw sample gradients at $x = 2$ are 1 or 3, each noisy around the full gradient 2. This is a pedagogical convex example illustrating *gradient-difference variance reduction*, not a theorem that every estimator becomes exact.

Core comparison: The older SPIDER analysis uses an accuracy-dependent, normalized step. SpiderBoost switches to an ordinary estimator-based update $x_{k+1} = x_k - \eta v_k$ with a much larger constant-order $\eta = 1/(2L)$.

2. From SpiderBoost to proximal momentum

Algorithm 1 and Theorem 1

Set an epoch length and a mini-batch size on the order of $\sqrt{n}$. SpiderBoost periodically evaluates a full gradient and otherwise applies recursive differences. Under a bounded-below finite-sum objective and L-smooth component losses, Theorem 1 bounds the number of stochastic first-order oracle calls by

$$O\left(n + \sqrt{n}\,\varepsilon^{-2}\right)$$

to obtain a random iterate satisfying $\mathbb{E}\|\nabla f(x)\| \le \varepsilon$, with scaling constants from the initial optimality gap suppressed.

Composite objectives: the proximal step

For $\Psi(x) = f(x) + h(x)$ with possibly nonsmooth regularizer h, use

$$x_{k+1} = \operatorname{prox}_{\eta h}(x_k - \eta v_k), \quad \operatorname{prox}_{\eta h}(z) = \arg\min_y \left\{ h(y) + \frac{\|y - z\|^2}{2\eta} \right\}.$$

The correct first-order criterion is now a *proximal-gradient mapping*, not necessarily $\|\nabla\Psi\|$ (which may be undefined). As a concrete example, if $h(x) = 0.3|x|$, $z = 0.8$ and $\eta = 1$, then $\operatorname{prox}_h(z) = 0.5$: the regularizer encourages sparsity by shrinking toward zero.

Prox-SpiderBoost-M (Algorithm 3)

The momentum variant keeps three related sequences x_k, y_k, z_k. It forms a convex-combination point z_k, calculates the same recursive estimator there, and takes two coupled proximal updates before recombining. Its extra mixing is not simply a fixed β term pasted onto SGD. Theorem 3 preserves the same near-optimal SFO order; the paper reports empirical acceleration in its experiments.

Reading priority: Section 2 (Eq. 1, Algorithms 1–2 and Theorems 1–2), Section 3 (Algorithm 3, Theorem 3), and Figures in Section 5. You can skim the detailed estimator-error proof.

3. Contributions, assumptions, and limits

Contributions. Large constant-level steps in SPIDER-type stochastic optimization; extension to composite nonsmooth regularization via prox; an efficient momentum construction with the same asymptotic first-order oracle order and better results on tested tasks. **Assumptions.** Finite-sum structure, lower-bounded objective, Lipschitz gradients of components, sampling and batch/epoch settings; the nonsmooth term needs a tractable proximal mapping. Conclusions are about expected generalized *first-order* stationarity. **Limitations.** No general second-order stationarity or saddle-escape theorem in this paper. The result does not claim global minima; gradient-evaluation complexity is distinct from runtime. Momentum gains are empirical despite unchanged asymptotic first-order complexity.

Why it matters alongside escape papers

SSRGD (Ref 83) and **Stabilized SVRG (Ref 82)** augment variance-reduced estimators with carefully designed perturbation/super-epoch mechanisms to seek second-order stationarity. **NEON2 (Ref 77)** instead supplies an explicit gradient-only negative-curvature detector. SpiderBoost supplies a fast first-order building block, *not* a completed saddle-escape mechanism.

Quick reference

Term	Explanation
v_k	Recursive gradient estimator with same-sample differences.
$\eta = 1/(2L)$	Representative constant-level step size under smoothness.
$\mathrm{prox}_{\eta h}$	Handles nonsmooth h, such as an ℓ_1 penalty.
Gradient mapping	Appropriate stationarity measure for composite objectives.
$O(n + \sqrt{n}\varepsilon^{-2})$	SFO order for first-order accuracy (with constants suppressed).

Papers Master note

SpiderBoost improves SPIDER/SARAH-style variance reduction via constant-order steps and develops proximal and momentum variants; it attains near-optimal first-order gradient complexity and reports empirical speedups. It does *not* itself prove second-order saddle escape.

Verified source: original NeurIPS 2019 PDF, Eq. (1), Algorithms 1–3, Theorems 1–3, Section 5.